

Opracowanie: OBL

Obliczenia w chmurze

HISTORIA:

- v. 1.00: 10.08.2018, JPok, przygotowanie opracowania zadania

dokument systemu SINOL 1.9.3

1 Treść zadania

Opracowujący zmienił treść zadania, aby nie pojawiała się w niej słowo "zadanie". Kojarzy się ono z zadaniem, które można zakończyć (w określonym czasie), co błędnie może naprowadzać na problemy szeregowania zadań na procesorach. W tym zadaniu przeznaczamy rdzeń raz na zawsze konkretnemu zleceniu.

Limit pamięci zmieniony na 64MB. Rozwiązanie wzorcowe zużywa bardzo mało pamięci.

2 Rozwiązania wzorcowe

W pliku `obl.cpp` jest oryginalne rozwiązanie wzorcowe dostarczone przez autora. W pliku `obl2.cpp` jest rozwiązanie opracowującego, które jest mniej efektywne – ma gorszą stałą. Różnica polega na operowaniu za każdym razem na tablicy DP pełnego rozmiaru, który jest rzędu 100 000, niezależnie od danych wejściowych. Rozwiązanie autora używa tylko tyle komórek, ile rzeczywiście potrzeba.

3 Rozwiązania błędne

- `obl1b1.cpp` Wzorcówka bez sortowania po częstotliwości. Przechodzi tylko grupę nr 5.
- `obl1b2.cpp` Metoda zachłanna kupująca procesory w kolejności od najtańszego w przeliczeniu na 1 rdzeń, a następnie po każdym zakupie próbuje wykonać jak najwięcej zadań. Zadania są również wybierane zachłannie od najdroższych w przeliczeniu na 1 rdzeń. Dostaje 0pkt.
- `obl1b3.cpp` Rozwiązanie wzorcowe używające zbyt małej tablicy DP, ma ona 10 005 komórek. Przechodzi grupy nr 1, 2 oraz 3.
- `obl1b4.cpp` Rozwiązanie wzorcowe z tablicą DP za małą o 1 komórkę. Przechodzi wszystkie grupy poza ostatnią (nr 6).
- `obl1b5.cpp` Wzorcówka bez long longów. Przechodzi tylko grupę nr 5.
- `obl1b6.cpp` Rozwiązanie zachłanne biorące procesory o najwyższej częstotliwości jako pierwsze i szukające `lower_bound`'em zlecenia możliwego do wynonania o możliwie największej częstotliwości. Rozwiązanie stworzone z myślą o podzadaniu nr 5. Dostaje 0pkt.

4 Testy

W większości są losowe, z odpowiednim doбором zakresów rdzeni, częstotliwości oraz kosztu – funkcje `SimpleGenTasks` oraz `SimpleGenProcessors`.

W grupie nr 1 zawarte są 2 testy skrajne $n = m = 1$, jeden gdzie się da wykonać zlecenie i drugi, gdzie się nie da – odpowiedź to 0.

Funkcje `GenTasks` oraz `GenProcessors` były używane do stworzenia "klonów" testów losowych – w większości grup jest seria testów generowanych losowo, do każdego testu zaraz po nim jest generowany jego "klon" metodą zawartą w tych magicznych funkcjach dostarczonych przez autora.

W niektórych grupach używana jest funkcja `GenFreqPriceProportional`, która rozmieszcza zlecenia i procesory losowo względem podanej (jako parametr) funkcji $f(x)$. Założenie jest takie, że wszystkie zlecenia i procesory wrzucamy do jednego wora, a następnie rysujemy wykres wartości (cena/nagroda) względem częstotliwości – ten wykres to właśnie funkcja $f(x)$. Obserwacja jest taka, że jeśli $\forall_i c_i = C_i = 1$ oraz f jest rosnąca, to nie opłaca się wykonywać żadnych zleceń. Jeśli natomiast jest malejąca, to nie opłaca się robić nic. W testach jest kilka funkcji jednego i drugiego typu. Niestety powyższa obserwacja nie jest prawdą bez założenia o liczbie rdzeni, niemniej jednak gdy wygenerujemy test z większą liczbą rdzeni na zadania i procesory, to trzeba zbierać "tańsze zadania, aby było nas stać na kupno droższego procesora (gdy f rosnąca), co też wydaje się być sensownym testem. Zawarte w grupie nr 6.

Test `obl6c.in` łączy w jednym teście zbiór zadań i procesorów, z których żaden się nie opłaca oraz część, gdzie wszystko się opłaca - stworzona jest tam funkcja nieciągła w tym celu.

5 Do rozważenia

Można wyrzucić/zmodyfikować niektóre duże testy, aby rozwiązanie bez long long'ów dostawało rozsądniejszą liczbę punktów – obecnie ma 18pkt przechodząc tylko grupę nr 5.

Dodać rozwiązanie bazujące na matchingu ważonym – sugerowane przez autora.